

ENPM680 – Introduction Secure Software Engineering Project (**DRAFT**)

Updated September 10, 2025

Please note: This is a draft guide for the Final Project for the course and is subject to change. Any changes will be communicated via ELMS.

Project Description

The goal of this project is to plan, design and implement a simple software web-based application that meets a set of requirements and specifications (as specified by you). This will be done in six phases.

Note on use of AI and ML based tools

You **ARE** allowed to use any of the AI or ML based tools such as Perplexity, Bard, ChatGPT etc. for this project!

This project will be conducted in **6 Phases**. Each phase has **strict** requirements and students going beyond the scope of each phase shall be penalized in their grades:

- Phase 1 – Setup of Git repository with strict naming convention along with a general description and use cases for your web-based application. **Due Friday, September 14, 2025.**
- Phase 2 – Requirements and Design Specifications with strict guidelines. **Due Sunday, September 28, 2025.**
- Phase 3 – Initial implementation of the requirements following the design specifications with strict guidelines on what can be implemented. **Due Sunday, October 26, 2025.**
- Phase 4 – Security Analysis and Secure Design update. **Due Sunday, November 9, 2025.**
- Phase 5 – Secure Design Implementation Update with addition of Error Handling and Logging. **Due Sunday, November 30, 2025.**
- Phase 6 – Secure Code Review by your peers. **Due Sunday, December 7, 2025.**

Project Phase 1 Guide: Application Description and Use Cases

(10% of your Final Project Grade)

Overview: create your Git repository and define your web application concept.

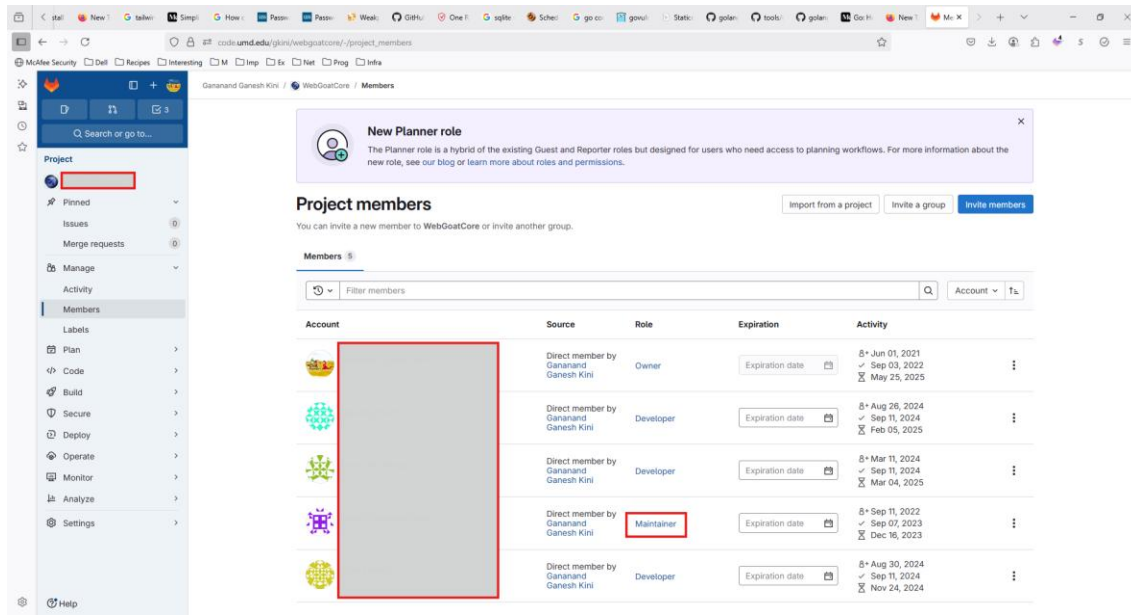
Phase 1 Part 1: Git Repository Setup

Repository Creation:

- Go to <https://code.umd.edu/> and sign in using UMD CAS and your UMD credentials.
- Click the Blue "New Project" button → "Create a Blank Project"
- Under Project URL, please select your UMD user ID as the namespace.
- **The Project name or slug must be:** ENPM680Fall25Project-<your-directory-id>
- Example: ENPM680Fall25Project-jsmith
- Project name is specified under the "Project slug" field.
- Leave default settings and click "Create Project".
- A new project with a default README.md file will be created and you will be seeing the README markdown file displayed in the web browser.
- If above instructions are not followed you will lose points.

Add Instructor and Grader Access:

- Go to "Manage" → "Members"
- Click "Invite Members"
- Add usernames: gkini, <TA>, <grader> with "Maintainer" role



Phase 1 Part 2: Project Folder on your Development Machine

- Create a project folder on your development machine called 'ENPM680Fall25Project-<Your UMD directory ID>'.
- Do NOT provide your project name here as the folder name. You will lose points if you do.
- Now use your Git skills to link the above code.umd.edu project to your locally created project folder. If in doubt, Gitlab provides instructions upon project creation on how to do it.
- Create an empty branch called 'Phase1' and push it to your project remote on code.umd.edu.
- If above instructions are not followed you will lose points.

Phase 1 Part 3: Application Description Document

Create a Word document in your local Project folder with the name: '[Your App Name] - Application Description and Use Cases.docx'

Required Information:

- Your name and UMD email
- Your UMD directory ID
- Application name (be creative!)

Phase 1 Part 3 Section 1: Application Description

Write a brief overview of what your web application will do. Think simple - something you

can build quickly in a few weeks. Here you may use Machine Learning/AI tools to help you derive ideas, however you will be required to describe the application yourself.

Examples of good applications:

- Library book management system
- Simple e-commerce store
- Blog system
- Social news aggregator
- Task management system

What NOT to include:

- Login/authentication features (these come later)
- Any security features
- Overly ambitious functionality

Phase 1 Part 3 Section 2: Use Cases

Describe how users will interact with your application. For each use case, include:

- **Actor:** Who is using the feature (librarian, customer, etc.).
- **Goal:** What they want to accomplish.
- **Success scenario:** Step-by-step happy path.
- **Error scenarios:** What happens when things go wrong.

Example Use Case:

- **Title:** Return a book
- **Actor:** Library patron
- **Goal:** Return a borrowed book
- **Steps:**
 - a. Patron scans book barcode
 - b. System finds book in database
 - c. System marks book as returned
 - d. System displays confirmation
- **Error cases:** If a Book is not found, system asks patron to talk to a Librarian. If a book has already been returned, system asks the patron if they would like to put a hold on it in the future.

Phase 1 Part 3 Section 3: Additional Requirements for the Application Description and Use Cases document:

Please ensure you have the following in your document:

- Table of Contents – A table of contents after the title page with page numbers that are links to the appropriate pages/sections. You can use Word feature to generate this.
- List of Figures – A table of figures included in the document. You can use Word feature to generate this as well.
- Application Description – Here you describe your application at a high level, include background information a lay reader would need to know to understand what your application does, along with references. This should not be just a sentence or two. This section should support the use cases you describe in the next section below.
- Use Cases – Here you describe all the different use cases your application would have. You need a minimum of 3 use cases here. But you should have more when you get to Phase 2.
- References section – A list of references using IEEE style that include background information about the topic you have chosen to implement for your final project.
- Include diagrams showing actor interactions along with the use cases.
- Do **not** show nor use any technology specific elements in this document! For example, do not say, this application uses a web server to serve up HTML and JavaScript using Reactjs and Vuejs etc. Those will be described in Phase 2 for Design specification.
- Do **not** show or use any security specific elements such as authentication, authorization, logging, error handling elements in this document.
- Focus on the functional happy path use cases.
- If above instructions are not followed you will lose points.

Submission

1. Commit the document '[Your App Name] - Application Description and Use Cases.docx' to the Phase1 branch.
2. Push Phase1 branch to your repository on code.umd.edu.
3. Upload a file named 'submission.txt' to Gradescope with:
 - Your UMD directory ID (first line)
 - Git commit ID (second line)

Project Phase 2 Guide: Requirements and Design (10% of your Final Project Grade)

Overview: Create your requirements document and create a detailed design for your application.

Prerequisites

- Completed Phase 1 with application description and use cases.
- Create a `Phase2` branch in your project Git repository.

Phase 2 Part 1: Enhanced Requirements Document

Create a new requirements document in the local project folder: `[Your App Name] - Application Requirements Specification.docx`

In this Phase, you will start thinking about error flows in addition to the functional happy path. Please ensure you have the following in your document:

Functional Requirements:

For each use case from Phase 1, create detailed functional requirements including:

- **Requirement ID:** FR-001, FR-002, etc.
- **Use Case Reference:** Link to specific use case.
- **Trigger:** What starts this function.
- **Preconditions:** What must be true before this functionality runs.
- **Success Flow:** Detailed step-by-step process.
- **Alternative Flows:** Different ways to accomplish the goal.
- **Postconditions:** What's true after completion.
- **Error Flows:** How errors are handled.

External Interface Requirements:

- How users will interact with your application (web browser, API, etc.).
- What data formats you'll use.
- Any external systems you'll connect to.

Include all potential requirements that make the application close to fully functional. This means for example, if you have a library system, just having a checkout functionality requirement, followed by book search is incomplete. The ability to add books to the system, remove books from the system, as well as return books in the system will make the application complete.

Phase 2 Part 2: Application Design Document

Create the following document in your local project folder: [Your App Name] - Application Design Specification.docx

Please ensure you have the following specified in your document:

Technology Stack:

Choose your programming language and frameworks:

- **Recommended:** Go Fiber v2 (course standard)
- **Alternatives:** Go, ASP.NET, Java, Python, or PHP (must be reviewable by peers)

Required Sections:

Application Overview:

- High-level description of your application
- Technology stack you'll use
- Overall architecture approach

System Architecture:

- How different components connect
- Data flow between components
- Simple diagrams showing relationships

Application Software Components:

- What classes/models you'll need
- Core business logic components
- How they relate to your use cases

Application UI Components:

- What pages/screens you'll have
- How users navigate between them
- Wireframes or mockups (optional but helpful)

Application Data Components:

- What data you need to store
- How data relates to each other
- Database tables or file structure

Test Cases:

- How you'll test each functional requirement

- What proves your design works
- Success criteria for each test

Traceability Matrix:

A table showing how your design components map to your requirements. This is a table that has requirements with IDs in rows and a column specifying the design components/classes/modules/packages that implement the requirement.

References:

You must include the following references:

1. IEEE Software Requirements Specification (See <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=720574>)
2. IEEE Software Design Descriptions (See <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=741934>)

Helpful Tools for showing the system architecture

- **Diagramming:** LucidChart, [Draw.io](https://draw.io)
- **Wireframing for UI:** Mockplus, [Wireframe.cc](https://wireframe.cc)

Submission

1. Add requirements document in Phase2 branch.
2. Add design document to Phase2 branch.
3. Push Phase2 branch to your repository on code.umd.edu.
4. Upload a file named 'submission.txt' to Gradescope with:
 - Your UMD directory ID (first line)
 - Git commit ID (second line)

Project Phase 3 Guide: Initial Implementation (20% of the Final Project Grade)

Overview: Build your application with basic functionality that meet the requirements and tests.

Prerequisites

- Completed requirements and design documents
- Create a `Phase3` branch in your project Git repository

Phase 3 Part 1: Implementation

Implement the software design to meet the requirements and follow the design constraints set in the earlier phase. Here you are allowed to use AI/ML tools to help you do vibe coding in order to quickly accomplish your software implementation goals. You are not required to add Error handling and meet the Error flows part of the requirements specification at this point. You need to have the following in your implementation:

Core Application:

- Implement all functional requirements from your design.
- Make sure your application actually works.
- You do not need to handle errors at this point yet.

Testing:

- Write unit tests for your core functionality:
 - **Java:** Use JUnit/Mockito
 - **C#/.NET:** Use xUnit
 - **Python:** Use pytest
 - **PHP:** Use PHPUnit
 - **Go:** Use Go's testing package

Documentation:

Create two additional documents in your local project folder with the given names:

Setup Guide ([Your App Name] – Build and Setup Guide.docx):

- How to install dependencies
- Database setup instructions
- How to build and run your application on another developer's machine from scratch
- Step-by-step instructions for developers

- **Prerequisites:** Required software, versions
- **Dependencies:** How to install libraries/frameworks
- **Database Setup:** Schema creation, initial data
- **Configuration:** Environment variables, config files
- **Build Process:** Step-by-step compilation/setup
- **Running:** How to start the application
- **Verification:** How to confirm it's working

User Guide ([Your App Name] - User Guide.docx):

- How to use each feature you've implemented
- Screenshots of your application
- How to exercise each use case
- Troubleshooting common issues

Please ensure you meet the following goals:

Code Organization

- Keep all source code in your project folder.
- Include any database scripts or configuration files.
- Organize code logically with clear folder structure.
- Add comments explaining complex logic.

Quality Requirements

- **Testing:** All major functionality must have tests.
- **Documentation:** Code should be well-commented.
- **Functionality:** Application must actually work as designed.

Submission

1. Add all code, tests, and documentation to Phase3 branch.
2. Ensure application builds and runs.
3. Push Phase3 branch to your repository on code.umd.edu.
4. Upload a file named 'submission.txt' to Gradescope with:
 - Your UMD directory ID (first line)
 - Git commit ID (second line)

Project Phase 4 Guide: Security Analysis (20% of your Final Project Grade)

Overview: Analyze your application for security weaknesses and plan security improvements.

Prerequisites

- Working application from Phase 3.
- Create a `Phase4` branch in your project Git repository.

Phase 4 Part 1: Security Requirements Analysis

Update your existing Requirements Document (`[Your App Name] - Application Requirements Specification.docx`) **with the following:**

Add Misuse Cases Section:

Think like an attacker - how could someone misuse your application?

For each misuse case, document:

- **Attacker:** Who might do this?
- **Goal:** What are they trying to achieve?
- **Method:** How would they attack?
- **Impact:** What damage could they cause?

Add STRIDE Threat Modeling Section:

Use the STRIDE framework to analyze threats:

- **Spoofing:** Impersonating users or systems
- **Tampering:** Modifying data or code
- **Repudiation:** Denying actions taken
- **Information Disclosure:** Exposing sensitive data
- **Denial of Service:** Making system unavailable
- **Elevation of Privilege:** Gaining unauthorized access

Add Security Requirements:

Based on your misuse cases and STRIDE threat modeling, add security requirements to your Non-Functional Requirements section:

- Input validation requirements
- Authentication/authorization needs

- Data protection requirements
- Error Handling needs
- Logging and monitoring needs

These will be similar to normal requirements, but now they will be addressing security goals instead of functional goals.

Part 2: Security Design Updates

Update your existing Design Document ([Your App Name] - Application Design Specification.docx) **with the following:**

Security Architecture:

- How will you validate user input?
- Where will you implement authentication/authorization?
- How will you protect sensitive data?
- What security controls will you add?

Logging Design:

Plan comprehensive logging for:

- User actions (who did what, when)
- Security events (failed logins, suspicious activity)
- System errors and exceptions
- Administrative actions

Security Components:

- Input validation modules
- Authentication/authorization components
- Encryption/data protection mechanisms
- Audit logging systems

Submission

1. Update both requirements and design documents in Phase4 branch.
2. Push Phase4 branch to your repository on code.umd.edu.
5. Upload a file named 'submission.txt' to Gradescope with:
 - Your UMD directory ID (first line)
 - Git commit ID (second line)

Project Phase 5 Guide: Secure Implementation (20% of your Final Project Grade)

Overview: Implement security features and create comprehensive documentation.

Prerequisites

- Security requirements and design changes from Phase 4.
- Create a `Phase5` branch in your project Git repository.

Implementation Updates

Security Features:

Implement the security requirements you defined in Phase 4:

- **Input Validation:** Sanitize and validate all user inputs.
- **Authentication/Authorization:** Control access to features.
- **Data Protection:** Encrypt sensitive data, secure data storage.
- **Error Handling:** Don't leak sensitive information in errors.
- **Logging:** Log security-relevant events.

Logging Implementation:

Your application must log:

- User authentication attempts (success/failure).
- Access to sensitive data or functions.
- Input validation failures.
- System errors and exceptions.
- Administrative actions.

Testing Updates:

- Ensure all existing tests still pass.
- Add tests for new security features.
- Test error handling and edge cases.
- Verify logging works correctly.

Documentation Requirements

Update your Build and Setup Guide (`[Your App Name] – Build and Setup Guide.docx`):

Complete instructions for setting up your application:

- **Prerequisites:** Required software, versions

- **Dependencies:** How to install libraries/frameworks
- **Database Setup:** Schema creation, initial data
- **Configuration:** Environment variables, config files
- **Build Process:** Step-by-step compilation/setup
- **Running:** How to start the application
- **Verification:** How to confirm it's working

Update your User Guide ([Your App Name] – User Guide.docx):

Comprehensive guide for end users:

- **Getting Started:** First-time setup, login process
- **Feature Walkthrough:** How to use each feature
- **Use Case Examples:** Step-by-step scenarios
- **Screenshots:** Visual guide to the interface
- **Troubleshooting:** Common problems and solutions
- **Security Features:** How security affects user experience

Quality Checklist

- ☐ All security requirements implemented
- ☐ Comprehensive logging in place
- ☐ All tests passing
- ☐ Error handling doesn't leak sensitive info
- ☐ Input validation on all user inputs
- ☐ Documentation is complete and accurate
- ☐ Application builds and runs from documentation

Submission

1. Add all updated code and documentation to `Phase5` branch.
2. Push `Phase5` branch to your repository on `code.umd.edu`.
6. Upload a file named `'submission.txt'` to Gradescope with:
 - Your UMD directory ID (first line)
 - Git commit ID (second line)

Project Phase 6 Guide: Secure Code Review (20% of your Final Project Grade)

Overview: Review another student's project for security vulnerabilities.

What You'll Receive

- Assignment of another student's project to review
- Read-only access to their Git repository
- Their requirements and design documents

Phase 6 Part 1: Oral Interview

Preparation:

- Read their requirements and design documents
- Understand what their application does
- Prepare questions about their implementation

Interview Questions:

Ask questions about:

- **Functionality:** What does the application do?
- **Setup:** How to build, run, and test it?
- **Security Categories:**
 - Input handling and validation
 - Cryptography usage
 - Authentication mechanisms
 - Authorization controls
 - Session management
 - Error handling approach
 - Logging implementation
 - Debug code presence
- **Assumptions:** What should you assume about the environment?

Take Notes: Document their answers and include an explanation in your report.

Phase 6 Part 2: Security Code Review

Review Process:

1. **Manual Code Review:** Read through their source code
2. **Automated Tools:** Use static analysis tools if available

3. **Testing:** Try to run their application and tests

Create a Findings Spreadsheet:

For each security issue found, document the CWE, Criticality, Security Category, Filename, Line number/position, Description describing the weakness and impact, Technical Impact chosen from cwe.mitre.org, Potential Mitigations and any Comments to the developer:

CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')	
Criticality	Critical
Security Category	Input Handling
Filename	WebGoat/Content/SqlInjection.aspx.cs
Line number/position	570
Description	The code on this line constructs a SQL query using user provided input without any sanitization leading to potential SQL Injection. On this page, an adversary is able to leak all the login email ids of customers by crafting a SQL injection input. This could lead to a loss of confidentiality for sensitive user's data in the application.
Technical Impact	Read Application Data – Note: Here the SQL injection could result in loss of confidentiality. This page does not allow for bypassing access control or other technical impacts.
Potential Mitigations	Consider using parameterized queries, sanitizing user inputs using regular expressions or encoding inputs ...
Comments	This seems to be a test page, and if so, it should be removed from the application before releasing it to production.

Criticality Guidelines:

- **Critical:** Could lead to complete system compromise
- **High:** Significant security impact, easy to exploit
- **Medium:** Moderate impact or harder to exploit
- **Low:** Minor security concern
- **Info:** Best practice violation, no immediate risk

Phase 6 Part 3: Security Review Report

Create a document: [Application Name] - Secure Code Review Report.docx

Required Sections:

Summary:

- Overview of findings by criticality.
- Mention any CWE Top 25 weaknesses found.
- Overall security assessment.

Introduction:

- Review methodology used.
- Developer information, Application Name, and code location.
- Tools used (manual review, static analysis, etc.).
- How you determined criticality levels.
- Limitations of your review.

Interview Section:

- Questions asked and key answers.
- Assumptions the developer requested.
- Notes that guided your review.

Code Review Findings:

- Detailed table of all findings.
- Group by weakness type or category.
- Sort by criticality (from Highest to Lowest).
- Include mitigation recommendations.

Impact Statement:

- What happens if findings aren't addressed?
- Business/organizational impact
- Risk assessment

References:

- CWE database references
- Security standards used
- Tools documentation

Submission

Upload your Word document or PDF of your report to Gradescope including:

- Your name and UMD email
- Student project you reviewed
- Complete report as outlined above

General Project Guidelines

Important Restrictions

- **AI/ML tools allowed** (Perplexity, ChatGPT, Bard, etc.) only for Phase 1 for idea generation and Phase 3 for software implementation.
- Must follow exact naming conventions
- All work must be individual
- Grading staff must have repository access

Grading Weight

- **Total Project:** 30% of course grade
- Each phase contributes to overall project success

Getting Help

- Ask questions before 6:00 PM the day before due dates.
- Use course discussion forums.
- Attend office hours for clarification.

Success Tips

- **Start early** - each phase builds on the previous
- **Keep it simple** - focus on core functionality
- **Document everything** - clear documentation is crucial
- **Test thoroughly** - make sure your application works
- **Follow security best practices** - this is a security course!

Grading Criteria and Rubric Used for Final Project (overall 30% of your Final Grade)

Phase 1: Application Description and Use Cases

Component	Requirements	Points Consideration
Git Repository Setup	- Correct naming: ENPM680Fall2024Project-<directory-id> - Branch named 'Phase1' - Instructor/TA/grader access granted	Critical: May receive 0 if not followed correctly
Application Description Document	- Word document: [Your App Name] - Application Description and Use Cases.docx - Correct naming as above. - Name, UMD email, directory ID - Application name and description - Table of Contents, List of Figures, References - Brief and detailed functionality description - No security features mentioned - No login/authentication requirements - Technology-agnostic language	Content quality, completeness, clarity and feasibility assessment
Use Cases	- Detailed use cases with actors, goals, success/error flows - Actor-based diagrams - Comprehensive coverage of functionality	Thoroughness and realistic scenarios

Phase 2: Application Requirements and Design

Component	Requirements	Points Consideration
Requirements Document	- Word document: [App Name] - Application Design Specification.docx - Correct naming as above. - Enhanced functional requirements - External interface requirements	Requirements traceability and detail

Component	Requirements	Points Consideration
	- Cross-references to use cases - Triggers, preconditions, postconditions	
Design Document	- Word document: [App Name] - Application Design Specification.docx - Correct naming as above. - Technology stack selection - System architecture overview	Design feasibility and clarity
Architecture Components	- Application software components - UI components description - Data components and relationships - Traceability matrix	Component relationships and mapping
Test Cases	- Test cases for each functional requirement - Success criteria definition - Design validation approach	Testing completeness and validity

Phase 3: Initial Implementation

Component	Requirements	Points Consideration
Working Application	- All functional requirements implemented - Application must actually work	Critical: Functionality must be demonstrable
Testing Implementation	- Unit tests for core functionality - Framework-appropriate testing (JUnit, xUnit, pytest, PHPUnit) - All tests must pass	Test coverage and quality
Documentation	- Build & Setup Guide (installation, dependencies, build/run) ([Your App Name] – Build and Setup Guide.docx) - User Guide (feature usage, screenshots, troubleshooting) ([Your App Name] – User Guide.docx) - Clear step-by-step instructions	Documentation completeness and clarity
Code Quality	- Organized folder structure - Well-commented code - All source code in Phase3 branch	Code organization and maintainability

Phase 4: Security Analysis

Component	Requirements	Points Consideration
Misuse Cases	- Security threat identification - Misuse and Abuse Cases are clearly spelled out - STRIDE threat modeling - Actor-based attack diagrams - Attacker goals and methods documented	Security analysis depth and accuracy
Security Requirements	- Security requirements added to Non-Functional Requirements - Input validation requirements - Authentication/authorization needs - Data protection requirements	Security requirement completeness
Updated Design for Security	- Security architecture integration - Logging design for audit trails - Security component specifications - Updated traceability matrix	Security design integration quality

Phase 5: Secure Implementation

Component	Requirements	Points Consideration
Security Features	- Input validation implementation - Authentication/authorization controls - Data protection mechanisms - Secure error handling	Security implementation effectiveness
Logging Implementation	- Security event logging - User action tracking - System error logging - Administrative action logs	Comprehensive audit trail capability
Updated Testing	- All existing tests still pass - Security feature testing - Error handling verification - Logging functionality tests	Test suite maintenance and expansion

Component	Requirements	Points Consideration
Complete Documentation	- Build and Installation Guide - Updated User Guide - Security feature documentation - Complete setup instructions	Documentation accuracy and completeness

Phase 6: Secure Code Review

Component	Requirements	Points Consideration
Oral Interview	- Interview conducted - Security category questions asked - Developer assumptions documented - Setup and functionality understanding	Interview thoroughness and preparation
Security Findings	- Excel spreadsheet with CWE-ID, security category, filename, line number - Detailed descriptions and technical impact - Appropriate criticality levels (Critical/High/Medium/Low/Info) - Mitigation recommendations	Finding quality and security expertise
Review Report	- [Application Name] - Secure Code Review Report.docx - Summary of findings by criticality - Introduction with methodology - Interview section with Q&A notes - Detailed findings table - Impact statement and references	Report professionalism and completeness
Review Quality	- CWE Top 25 identification - Appropriate criticality assessment - Actionable mitigation advice - Professional communication	Security review expertise demonstration